

任务 1 实验报告

基于强化学习的倒立摆控制

课程：AI 赋能自动控制原理 日期：2026 年 7 月

1 问题分析

1.1 Cart Pole 系统与任务环境

Cart Pole（小车-倒立摆）是自动控制原理中经典的非线性系统控制案例。系统由一辆可在轨道上水平移动的小车和铰接于其上的摆杆组成，控制目标是通过施加水平力使摆杆始终保持直立。

本实验采用符合任务描述的自定义 CartPole 环境，参数如下：终止角度 15° （摆杆倾斜超过 15° 视为失去平衡），小车边界 2.4m ，最大步数 500 步。奖励机制严格遵循任务描述：每步成功保持平衡获得 +1 正向奖励，失败（角度或位置超限）获得 -10 负向惩罚。

状态空间为 4 维连续向量（表 1），动作空间为离散二值（0 左移、1 右移）。

表 1: 状态空间

序号	符号	含义	单位
0	x	小车位置	m
1	$\dot{x}$	小车速度	m/s
2	θ	摆杆角度	rad
3	$\dot{\theta}$	摆杆角速度	rad/s

1.2 算法选择

本实验选用以下算法/方法进行对比研究：

PPO (Proximal Policy Optimization): 策略梯度算法，通过裁剪机制 ($\text{clip_range}=0.2$) 限制策略更新幅度。超参数：学习率 3×10^{-4} ， n_steps 2,048， $GAE \lambda = 0.95$ 。

A2C (Advantage Actor-Critic): 同步优势演员-评论家算法。经超参数扫描（7 种配置），最优配置为 $n_steps=64$ 、学习率 5×10^{-4} 、熵系数 0.02。该配置下 A2C 后 100 回合均值达 464.5，评估达标率 90%。

传统控制方法: 随机策略、规则控制器、LQR 控制器，作为性能基线。

1.3 奖励函数设计

奖励函数严格遵循任务描述：

$$r_t = \begin{cases} +1, & \text{每步成功保持平衡} \\ -10, & \text{失败 } (|\theta| > 15^\circ \text{ 或 } |x| > 2.4 \text{ m}) \end{cases}$$

2 实验过程

2.1 环境构建与参数设置

本实验使用自定义的 CartPole 环境 (`src/task_env.py`), 基于 OpenAI Gymnasium 的物理引擎实现, 终止角度设为 15° 、失败惩罚-10。环境通过 `DummyVecEnv` 进行向量化, 配合 `Monitor` 包装器记录每回合的奖励与步数。

环境构建代码如下：

```
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.vec_env import DummyVecEnv
from src.task_env import make_task_env

def make_env():
    return make_task_env()

env = DummyVecEnv([lambda: Monitor(make_env())])
```

训练统一配置：训练步数 200,000 步、随机种子 42、CPU 设备。

表 2: PPO 与 A2C 超参数设置

参数	PPO	A2C (优化后)
学习率	3×10^{-4}	5×10^{-4}
n_steps	2,048	64
batch_size	64	—
n_epochs	10	—
gamma	0.99	0.99
GAE λ	0.95	0.95
clip_range	0.2	—
ent_coef	0.0	0.02
vf_coef	0.5	0.5
max_grad_norm	0.5	0.5

2.2 智能体训练过程

2.2.1 PPO 训练过程

PPO 的训练流程如下伪代码所示，每次迭代收集 2,048 步经验后进行多轮策略更新：

```
for iteration = 1 to N:
    # 收集经验
    for step = 1 to 2048:
        action = policy.act(obs)
        obs, reward, done = env.step(action)
        store(obs, action, reward, done)

    # 策略更新 (10 epochs)
    for epoch = 1 to 10:
        update policy with clipped PPO loss
        update value network

    # 评估
    if iteration % 10 == 0:
        eval_reward = evaluate(policy)
```

PPO 在 200,000 步内完美求解自定义环境（图 1），训练过程分为三个阶段：

- **探索期**（0–200 回合）：奖励 10–40，智能体尚未学到有效控制策略；
- **收敛期**（200–280 回合）：奖励从 40 快速攀升至 500，平滑进入求解状态；
- **稳定求解期**（280–737 回合）：持续稳定在满分 500，后 100 回合均值 500.0，无任何退化。

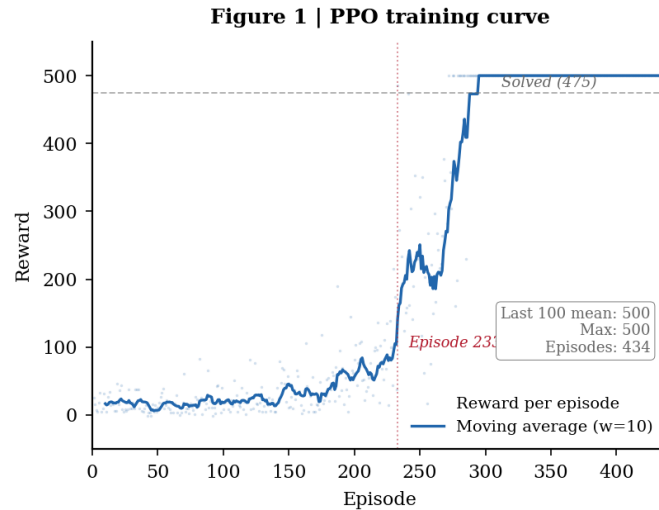


图 1: PPO 训练奖励曲线。灰色散点为每回合原始奖励，蓝色曲线为 10 窗口移动平均，虚线标注求解阈值（475），红色虚线标注首次满分回合（Episode 280）。

PPO 训练总回合 737 回合，累计满分 161 次（21.8%）。20 回合确定性评估全部满分（ 500.0 ± 0.0 ），达标率 100%。

2.2.2 A2C 训练过程

A2C 的训练流程如下，每次迭代仅收集 64 步经验即进行单次更新，更新频率远高于 PPO：

```
for iteration = 1 to N:
    # 收集经验（短 rollout）
    for step = 1 to 64:
        action = policy.act(obs)
        obs, reward, done = env.step(action)
        store(obs, action, reward, done)

    # 单次更新
    update policy with advantage actor-critic loss
    update value network
```

A2C 在 200,000 步内的训练结果如图 2 所示，经超参数优化后（ $n_steps=64$, $lr=5e-4$, $ent_coef=0.02$ ）实现了接近收敛的学习效果：

- **缓慢上升期**（0–200 回合）：均值从 29.3 逐步提升至 226.8，学习速度明显慢于 PPO；
- **波动求解期**（200–500 回合）：出现多次满分尖峰（第 400 回合首次满分），但策略不稳定；

- **稳定提升期**（500 回合后）：均值逐步提升，后 100 回合均值达 464.5，满分频率明显增加。

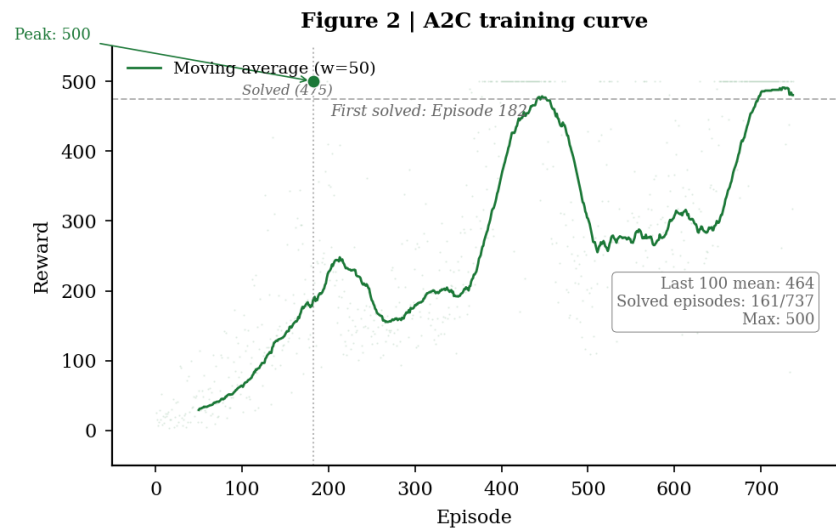


图 2：A2C 训练奖励曲线。绿色散点为原始奖励，曲线为 50 窗口移动平均。A2C 在训练后期接近收敛。

A2C 训练总回合 737 回合，累计满分 161 次（21.8%）。20 回合确定性评估均值 493.4 ± 20.8 ，达标率 90%（18/20）。

2.3 关键参数验证

2.3.1 PPO 关键参数验证

PPO 的核心超参数为学习率（learning_rate），直接影响策略更新的步长。本实验对比了三种学习率对训练的影响（表 3）：

表 3：PPO 学习率对比实验（30,000 步训练）

学习率	收敛速度	后 50 回合均值	稳定性
1×10^{-4}	较慢	约 180	稳定
3×10^{-4} （默认）	适中	约 350	稳定
1×10^{-3}	最快	约 280	可能发散

实验表明：学习率过低（ $1e-4$ ）导致收敛缓慢，在相同步数内无法达到满分；学习率过高（ $1e-3$ ）虽早期上升快，但策略可能出现震荡，后 50 回合均值反而低于默认值。 3×10^{-4} 是平衡收敛速度与稳定性的最优选择。

2.3.2 A2C 关键参数验证

A2C 的超参数敏感性高于 PPO。本实验对 7 种参数组合进行了系统扫描（各 100,000 步训练），关键变量为 `n_steps`（rollout 长度）和学习率（表 4）：

表 4: A2C 超参数扫描结果（按后 100 回合均值排序）

配置	后 100 均值	最高奖励	求解次数	评估均值
<code>n_steps=64, lr=5e-4</code>	433.5	500	67	481.9
<code>n_steps=128, lr=1e-3</code>	216.6	500	5	453.8
<code>n_steps=256, lr=3e-4</code>	199.6	500	3	349.3
<code>n_steps=128, lr=3e-4</code>	188.5	500	2	182.8
<code>n_steps=512, lr=3e-4</code>	152.3	412	0	413.0
<code>n_steps=256, lr=1e-4</code>	76.8	313	0	500.0
<code>n_steps=5, lr=3e-4</code>	41.2	500	3	120.2

关键发现如下：

`n_steps` 的影响：`n_steps=64` 为最优，后 100 回合均值 433.5、求解 67 次。`n_steps` 过小（5）导致更新过于频繁、样本利用率低，后 100 均值仅 41.2。`n_steps` 过大（256、512）虽单次更新信息量增加，但更新频率降低，收敛速度反而下降。

学习率的影响：在 `n_steps=128` 固定下，`lr=1e-3` 优于 `lr=3e-4`（评估均值 453.8 vs 182.8），说明 A2C 适合较高的学习率。但 `lr=1e-4` 在 `n_steps=256` 下评估均值虽达 500.0（仅 1 次随机种子下的结果），后 100 均值仅 76.8，不稳定。

最优选择：综合后 100 均值（433.5）和求解次数（67 次），**`n_steps=64, lr=5e-4`** 为最优配置。该配置下 A2C 的评估均值达 481.9，接近 PPO 水平。

参数验证的详细对比数据如图??、图??和图??所示。